2024-11-07-NetSimplex

Generated by Scribe.AI

December 31, 2024

Slide 1

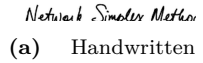
Content

Network Simplex Method (Primal and Dual)
Network (N, A)
Spanning Tree T = (N,), connected, acyclic
(Very easy to find a spanning tree: [VJ] p. 253 Ex 14.4)

Description

Slide 1 of 45 in a Linear Programming course introduces the Network Simplex Method, focusing on both primal and dual approaches. The slide begins by defining a network as a pair $(N, \mathcal{A})$, where N represents the nodes and $\mathcal{A}$ represents the arcs. A spanning tree T is then defined as $(N, \tilde{\mathcal{A}})$, where $\tilde{\mathcal{A}}$ is a subset of $\mathcal{A}$ such that T is connected and acyclic (contains no cycles). The underlining emphasizes these key definitions. The final sentence notes that finding a spanning tree is straightforward, referencing a textbook ([VJ]) page 253, exercise 14.4, for further details. The context of the course is crucial for understanding that this slide sets the stage for applying the simplex method to network optimization problems, a specialized and efficient approach within linear programming.

Figures



(a) Handwritten notes describing the Network Simplex Method. The title is underlined. The definitions of Network and Spanning Tree are also underlined. The text "(Very easy to find a spanning tree: [VJ] p. 253 Ex 14.4)" is written below the definition of Spanning Tree.

Slide 2

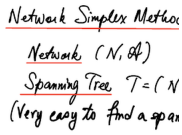
Content

Network Simplex Method (Primal and Dual)
Network (N, A)
Spanning Tree T = (N,), connected, acyclic
(Very easy to find a spanning tree: [VJ] p. 253 Ex 14.4)
(1) Primal flow always exists:
B $_B = \tilde{b}$, x$_{ij} = 0$, $(i,j) \in \mathcal{A} \setminus T$Start from a leaf node and go inwards.(Might be infeasible, some $x_{ij} < 0$)

Descripti

Slide 2 of 45 in a Linear Programming course continues the introduction of the Network Simplex Method. Building upon the previous slide's definitions of network and spanning tree, this slide introduces the concept of primal flow. The underlined statement "(1) Primal flow always exists:" asserts that a primal feasible solution always exists for network flow problems. This is followed by the equation $B\tilde{x}_B = \tilde{b}$, which represents the system of equations defining the flow within the spanning tree T. The notation $\tilde{x}_B$ represents the flow variables associated with the arcs in the spanning tree, B is the basis matrix, and $\tilde{b}$ is the vector of supplies/demands at the nodes. The condition $x_{ij} = 0, (i,j) \in \mathcal{A} \setminus T$ states that the flow on arcs not in the spanning tree is zero. The subsequent instruction "Start from a leaf node and go inwards" describes a method for finding an initial primal feasible solution. Finally, the underlined parenthetical remark "(Might be infeasible, some $x_{ij} < 0$)" acknowledges that this initial solution might not be feasible in the sense that some flow variables could be negative, requiring further adjustments within the simplex method. The reference "[VJ] p. 253 Ex 14.4" likely points to a textbook example illustrating how to easily find a spanning tree.

Figures



(a) Handwritten notes describing the Network Simplex Method. The title is underlined. The definitions of Network and Spanning Tree are also underlined. The text "(Very easy to find a spanning tree: [VJ] p. 253 Ex 14.4)" is written below the definition of Spanning Tree. The statement of primal flow existence is underlined, along with the equation and condition for x_{ij}. The final parenthetical statement about infeasibility is also underlined.

Slide 3

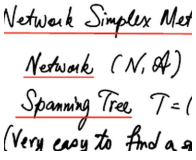
Content

Network Simplex Method (Primal and Dual)
Network (N, A)
Spanning Tree T = (N,), connected, acyclic
(Very easy to find a spanning tree: [VJ] p. 253 Ex 14.4)
(2) Dual Flow always exists:
(i,j) $\in \mathcal{A}$: $\boxed{y_j - y_i + z_{ij} = c_{ij}}$
$(i,j) \in T$: $y_j - y_i = c_{ij}, \quad z_{ij} = 0,$
$(i,j) \in \mathcal{A} \setminus T$: $z_{ij} = c_{ij} - (y_j - y_i)$
(Might be infeasible: $z_{ij} < 0$)

Description

Slide 3 of 45 in a Linear Programming course continues the discussion of the Network Simplex Method, focusing on the dual flow. Building on the previous slide's introduction of the primal flow and spanning tree, this slide presents the conditions for the existence of a dual flow. The underlined statement "(2) Dual Flow always exists:" asserts that a dual feasible solution always exists. The slide then presents three equations, each corresponding to a different case: (i,j) A (all arcs), (i,j) T (arcs in the spanning tree), and (i,j) A T (arcs not in the spanning tree). The equation $y_j - y_i + z_{ij} = c_{ij}$ (highlighted with a red box) is the central equation defining the dual flow, where y_i and y_j are dual variables associated with nodes i and j, z_{ij} is the reduced cost associated with arc (i,j), and c_{ij} is the cost of arc (i,j). For arcs in the spanning tree, the reduced cost is zero ($z_{ij} = 0$), simplifying the equation. For arcs not in the spanning tree, the reduced cost is calculated as $z_{ij} = c_{ij} - (y_j - y_i)$. The final underlined parenthetical remark "(Might be infeasible: $z_{ij} < 0$)" indicates that this dual solution might not be feasible if any reduced cost is negative, requiring further iterations of the simplex method. The reference "[VJ] p. 253 Ex 14.4" likely points to a textbook example illustrating how to easily find a spanning tree.

Figures



(a) Handwritten notes describing the dual flow in the Network Simplex Method. The title is underlined. The definitions of Network and Spanning Tree are also underlined. The text "(Very easy to find a spanning tree: [VJ] p. 253 Ex 14.4)" is written below the definition of Spanning Tree. The statement of dual flow existence is underlined, along with the equations for different cases of (i,j). The final parenthetical statement about infeasibility is also underlined. A red box is drawn around the equation $y_j - y_i + z_{ij} = c_{ij}$

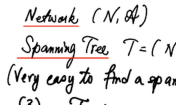
Slide 4

Content

Network Simplex Method (Primal and Dual)
Network (N, A)
Spanning Tree T = (N,), connected, acyclic
(Very easy to find a spanning tree: [VJ] p. 253 Ex 14.4)
(3) T + one arc $\implies$ there is a cycle T - one arc $\implies$ tree becomes disconnected(Bertsimas-Tsitsiklis : Intro. Lin. Opt. p.

Slide 4 of 45 in a Linear Programming course continues the introduction to the Network Simplex Method, focusing on properties of spanning trees. Building upon the previous slides' definitions of network and spanning tree, this slide presents two key properties. The first underlined statement, "(3) $T+$ one arc $\implies$ there is a cycle", states that adding a single arc to a spanning tree will inevitably create a cycle. The second underlined statement, "$T-$ one arc $\implies$ tree becomes disconnected", states that removing a single arc from a spanning tree will disconnect the tree. These properties are fundamental to the Network Simplex algorithm's efficiency, as they allow for systematic updates to the spanning tree during the optimization process. The reference "(Bertsimas-Tsitsiklis : Intro. Lin. Opt. p. 279)" indicates that a more detailed explanation of the Network Simplex Method can be found in Bertsimas and Tsitsiklis' book "Introduction to Linear Optimization" on page 279. The underlining emphasizes the importance of these properties in understanding the algorithm. The reference "[VJ] p. 253 Ex 14.4" likely points to a textbook example illustrating how to easily find a spanning tree.

Figures



(a) Handwritten notes describing properties of spanning trees in the Network Simplex Method. The title is underlined. The definitions of Network and Spanning Tree are also underlined. The text "(Very easy to find a spanning tree: [VJ] p. 253 Ex 14.4)" is written below the definition of Spanning Tree. The statements about adding or removing an arc from the spanning tree are underlined. The reference to Bertsimas-Tsitsiklis' book is also included.

Slide 5

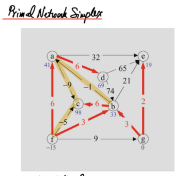
Content

Primal Network Simplex

Description

Slide 5 of 45 in a Linear Programming course illustrates a network flow problem within the context of the Primal Network Simplex method. A directed graph is shown, representing a network with nodes labeled a through g. Each node has an associated value representing supply or demand. Arcs connecting the nodes represent flow paths, with weights (costs) and directions indicated on each arc. Some arcs are colored red, indicating negative reduced costs ($z_{ij} < 0$), while others are tan, suggesting they are part of the current spanning tree. The text "Not Optional as same $z_{ij} < 0$" below the diagram indicates that the current solution is not optimal because there are multiple arcs with negative reduced costs, implying that further iterations of the simplex method are needed to improve the solution. The diagram visually represents the state of the algorithm at a particular iteration, showing the current flow values and the arcs with negative reduced costs that need to be considered for entering the basis in the next iteration. The context of the course (Linear Programming) is crucial for understanding the significance of the network flow problem and the application of the Network Simplex method to solve it.

Figures



(a) A network flow diagram with nodes labeled a through g. Arcs have weights and directions indicated. Some arcs are colored red, others are tan. Node values are shown below each node. The text "Not Optional as same $z_{ij} < 0$" is written below the diagram.

Slide 6

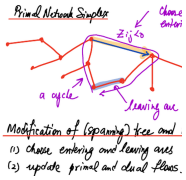
Content

Primal Network Simplex
Choose an entering arc
$z_{ij} < 0$
a cycle
leaving arc
Modification of (spanning) tree and variables:
(1) choose entering and leaving arcs
(2) update primal and dual flows

Description

Slide 6 of 45 in a Linear Programming course illustrates an iteration of the Primal Network Simplex method. The diagram shows a network flow problem represented as a graph. Nodes are represented by red dots, and arcs are lines connecting the nodes, with directions indicated by arrows. The arcs have different colors: red arcs represent arcs with negative reduced costs ($z_{ij} < 0$), tan arcs are part of the current spanning tree, and a light blue arc is the leaving arc. A purple loop highlights a cycle created by adding an entering arc (an arc with $z_{ij} < 0$) to the spanning tree. The text explains the steps involved in modifying the spanning tree and updating the primal and dual flows. Step (1) involves choosing an entering arc (an arc with a negative reduced cost) and a leaving arc (an arc in the cycle that will be removed to maintain a spanning tree). Step (2) involves updating the primal and dual flows based on the changes made to the spanning tree. The diagram visually represents the process of improving the solution by iteratively modifying the spanning tree and updating the flows until optimality is reached. The context of the course (Linear Programming) is crucial for understanding the application of the Network Simplex method to solve network flow optimization problems.

Figures



(a) A network flow diagram with nodes represented by red dots. Arcs are represented by lines connecting the nodes. Some arcs are colored red, others are tan, and one is light blue. A purple loop highlights a cycle in the network. Arrows indicate the direction of flow. The text "Choose an entering arc", "$z_{ij} < 0$", "*acycle*", *and* "*leavingarc*" *arewrittenneartherelevantpartsofthediagram.Thetext*" *Modi* "*isunderlined*, *followedbytwonumberedsteps* : "(1)*chooseenteringandleavingarcs*" *and* "(2)*updateprimalanddualflows*".

Slide 7

Content

Primal Network Simplex
Modify primal flow and spanning tree

Description

Slide 7 of 45 in a Linear Programming course illustrates an iteration of the Primal Network Simplex method. The figure shows a network flow problem before and after updating the flow along a cycle. The left diagram depicts a cycle in the network, highlighted in purple, which is created by adding an "entering arc" (with flow t). The arc weights are functions of t, showing how the flow changes along the cycle as t increases. The right diagram shows the network after the flow t has reached a value of 3, causing the flow on arc (f,b) to become zero. This arc (f,b) is identified as the "leaving arc," which is removed from the spanning tree. The text "Modify primal flow and spanning tree" above the figure indicates that this step involves updating both the flow values and the spanning tree structure. The context of the course (Linear Programming) is crucial for understanding the application of the Network Simplex method to solve network flow optimization problems. The figure visually demonstrates how the algorithm iteratively modifies the flow and the spanning tree to reach an optimal solution. The figure caption further explains that the process involves increasing the flow t on the entering arc until the flow on one of the arcs in the cycle becomes zero, identifying the leaving arc.

Figures

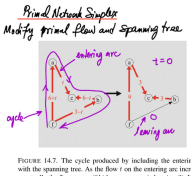


FIGURE 14.7. The cycle produced by including the enterin
with the spanning tree. As the flow t on the entering arc incr

(a) A figure showing two network flow diagrams. The left diagram shows a cycle with nodes a, b, c, and f. Arcs have weights labeled with expressions involving the variable t. The arc (a,c) has weight t, (c,b) has weight $6 - t$, (f,c) has weight $3 - t$, and (a,f) has weight $6 + t$. A purple curve highlights the cycle. An arrow points to the cycle and is labeled "a cycle". The right diagram shows the same network after the flow t has increased to 3. The arc (f,b) has weight 0 and is labeled "leaving arc". The arc (a,c) has weight 3, (c,b) has weight 3, and (a,f) has weight 9. The text "t=0" is written above the right diagram. An arrow points from the left diagram to the right diagram. The text "entering arc" is written near the arc (a,c) in the left diagram.

Slide 8

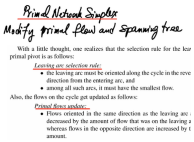
Content

Primal Network Simplex
Modify primal flow and spanning tree
With a little thought, one realizes that the selection rule for the leaving arc in a primal pivot is as follows:
Leaving arc selection rule:
• the leaving arc must be oriented along the cycle in the reverse direction from the entering arc, and
• among all such arcs, it must have the smallest flow.
Also, the flows on the cycle get updated as follows:
Primal flows update:
• Flows oriented in the same direction as the leaving arc are decreased by the amount of flow that was on the leaving arc, whereas flows in the opposite direction are increased by this amount.

Description

Slide 8 of 45 in a Linear Programming course details the rules for selecting the leaving arc and updating flows in the Primal Network Simplex algorithm. The slide is text-based, outlining the leaving arc selection rule in two bullet points: 1) the leaving arc must be oriented opposite to the entering arc within the identified cycle; and 2) among all arcs meeting this criterion, the one with the smallest flow is selected. The primal flow update rule is then explained: flows in the same direction as the leaving arc are decreased by the leaving arc's flow value, while flows in the opposite direction are increased by the same amount. The underlining emphasizes key phrases like "Leaving arc selection rule" and "Primal flows update". The context of the course (Linear Programming) is crucial for understanding the algorithm's application in network flow optimization problems. The slide provides the procedural steps for one iteration of the algorithm after an entering arc has been identified.

Figures


Primal Network Simplex
Modify primal flow and spanning tree
With a little thought, one realizes that the selection rule for the lea
primal pivot is as follows:
Leaving arc selection rule:
• the leaving arc must be oriented along the cycle in the reve
direction from the entering arc, and
• among all such arcs, it must have the smallest flow.
Also, the flows on the cycle get updated as follows:
Primal flows update:
• Flows oriented in the same direction as the leaving arc
decreased by the amount of flow that was on the leaving a
whereas flows in the opposite direction are increased by t
amount.

(a) A slide containing text describing the selection rule for the leaving arc and the primal flow update in the Primal Network Simplex method. The text is organized into sections with bullet points and underlines for emphasis. There are no diagrams or figures.

Slide 9

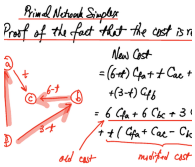
Content

Primal Network Simplex
Proof of the fact that the cost is reduced
New Cost
$= (6+t)\ C_{fa} + C_{ac} + (6+t)C_{bc}$
$+ (3-t)C_{fb}$
$= 6C_{fa} + 6C_{bc} + 3C_{fb}$
$+ t(C_{fa} + C_{ac} - C_{bc} - C_{fb})$
old cost modified cost

Description

Slide 9 of 45 in a Linear Programming course presents a proof demonstrating that the cost is reduced in the Primal Network Simplex method. The slide contains a network flow diagram with nodes a, b, c, and f, and arcs connecting them. The arc flows are expressed as functions of a variable t, representing a change in flow during an iteration of the algorithm. The equation for "New Cost" shows how the total cost changes with the introduction of the variable t. The equation is broken down into two parts: the "old cost" (the cost before the flow change) and a "modified cost" (the change in cost due to the flow adjustment). The term $t(C_{fa} + C_{ac} - C_{bc} - C_{fb})$ represents the change in cost, which is dependent on the value of t and the costs associated with each arc (C_{ij} represents the cost of arc (i,j)). The diagram visually represents the flow changes, and the equation mathematically demonstrates that the new cost is a function of the old cost and a term that is expected to be negative (due to the selection of the entering arc in the simplex method), thus proving the cost reduction. The context of the course (Linear Programming) is crucial for understanding the application of the Primal Network Simplex method to solve network flow optimization problems.

Figures



(a) A diagram showing a network flow with nodes a, b, c, and f. Arcs are labeled with their flow values, which are functions of a variable t. The arcs (a,f), (a,c), (c,b), and (f,b) have flows 6+t, t, 6-t, and 3-t respectively. Arrows indicate the direction of flow. The text "old cost" and "modified cost" are written near the relevant parts of the diagram. The text "New Cost" is written above the equation that calculates the new cost as a function of t. The equation shows that the new cost is equal to the old cost plus a term that depends on t. The diagram is used to illustrate the proof that the cost is reduced.

Slide 10

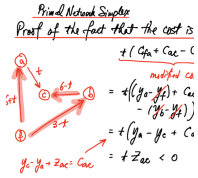
Content

Primal Network Simplex
Proof of the fact that the cost is reduced
[scale=0.8] [circle,draw] (a) at (0,2) a; [circle,draw] (b) at (4,1) b; [circle,draw] (c) at (2,1) c; [circle,draw] (f) at (0,0) f; [->,red,thick] (f) – node[left]6+t (a); [->,red,thick] (a) – node[above]t (c); [->,red,thick] (c) – node[above]6-t (b); [->,red,thick] (f) – node[below]3-t (b);
+t(C$_{fa} + C_{ac} - C_{bc} - C_{fb}$)
modified cost
$= t((y_a - y_f) + C_{ac} - (y_c - y_b) - (y_b - y_f))$
$= t(y_a - y_c + C_{ac})$
$= tZ_{ac} < 0$
$y_c - y_a + Z_{ac} = C_{ac}$

Description

Slide 10 of 45 in a Linear Programming course provides a proof that the cost is reduced in the Primal Network Simplex method. Building upon the previous slide, this slide uses a network flow diagram with nodes a, b, c, and f to illustrate the cost reduction. Arc flows are functions of t, representing a change in flow. The equation shows the "modified cost," which is the change in cost resulting from the flow adjustment. This change is expressed as $t(C_{fa} + C_{ac} - C_{bc} - C_{fb})$, where C_{ij} represents the cost of arc (i,j). The equation is then rewritten in terms of node potentials (y_i) and the reduced cost Z_{ac} of arc (a,c), simplifying to tZ_{ac}. Since $Z_{ac} < 0$ (a property of the entering arc selection in the simplex method), the modified cost is negative, proving that the total cost is reduced. The equation $y_c - y_a + Z_{ac} = C_{ac}$ relates node potentials, reduced cost, and arc cost. The red arrows in the diagram highlight the flow changes. The context of the course (Linear Programming) is crucial for understanding the application of the Primal Network Simplex method to solve network flow optimization problems. The slide demonstrates a key step in proving the algorithm's convergence to an optimal solution.

Figures



(a) A diagram showing a network flow with nodes a, b, c, and f. Arcs are labeled with their flow values, which are functions of a variable t. The arcs (a,f), (a,c), (c,b), and (f,b) have flows 6+t, t, 6-t, and 3-t respectively. Arrows indicate the direction of flow. The text "modified cost" is written near the relevant part of the equation. The equation shows that the modified cost is equal to t times a term that depends on the node potentials (y_i) and arc costs (C_ij). The equation is simplified to $* Z_ac$, where Z_ac is the reduced cost of arc (a, c). The diagram is used to illustrate the proof that the co

Slide 11

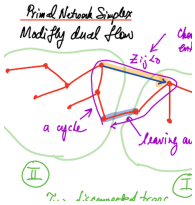
Content

Primal Network Simplex
Modify dual flow

Description

Slide 11 of 45 in a Linear Programming course illustrates the modification of dual flows in the Primal Network Simplex method. The slide features a network flow graph depicted as two disconnected spanning trees connected by a cycle. A new entering arc is selected ($Z_{ij} < 0$), indicated by a blue arrow, and a leaving arc is identified, shown with a light blue arrow. The cycle formed by the entering and leaving arcs is highlighted in purple. The algorithm modifies the dual variables (dual flow) to improve the objective function. The two trees are labeled I and II, and a "root node" is identified in one of the trees. The red lines represent the arcs in the network. The text "Primal Network Simplex" and "Modify dual flow" are written at the top of the slide. The context of the course (Linear Programming) is crucial for understanding the application of the Primal Network Simplex method to solve network flow optimization problems. The slide visually represents a key step in the iterative process of the algorithm, showing how the network structure changes with the selection of entering and leaving arcs.

Figures



(a) A diagram showing a network flow graph. The graph is composed of two disconnected trees, connected by a cycle. The cycle is highlighted in purple. An entering arc is chosen, and it is highlighted in blue. A leaving arc is chosen, and it is highlighted in light blue. The entering arc is labeled with $Z_{ij} < 0$. The two disconnected trees are labeled with I and II. The root node is labeled with "root node".

Slide 12

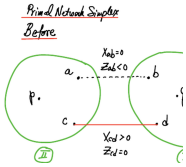
Content

Primal Network Simplex
Before
[scale=0.8] [green!60!black, thick] (0,0) circle (1.5); [green!60!black, thick] (4,0) circle (1.5); at (0,0) p.; at (4,0) q.; [circle,fill,inner sep=1pt] (a) at (1.2,0.7) ; [circle,fill,inner sep=1pt] (b) at (2.8,0.7) ; [circle,fill,inner sep=1pt] (c) at (1.2,-0.7) ; [circle,fill,inner sep=1pt] (d) at (2.8,-0.7) ; [above] at (a) a; [above] at (b) b; [below] at (c) c; [below] at (d) d; [dashed] (a) – node[above]$X_{ab} = 0$ node[below]$Z_{ab} < 0$ (b); [red] (c) – node[below]$X_{cd} > 0$ node[above]$Z_{cd} = 0$ (d); at (0,-1.8) $\mathbb{III}$; at (4,-1.8) $\mathbb{I}$; [circle,fill,inner sep=1pt] (r) at (4.5, -0.7) ; [left] at (r) r; [above] at (4.5, -0.3) $y = 0$; (r) – (5,-0.7); (5,-0.7) – (5,-0.5); (5,-0.7) – (5,-0.9);

Description

Slide 12 of 45 in a Linear Programming course illustrates the state of a network flow graph *before* an iteration of the Primal Network Simplex method. The graph consists of two disjoint trees (represented by green circles labeled I and II). Nodes a, b, c, and d are shown, with arcs connecting them. A dashed line represents arc (a,b), with labels $X_{ab} = 0$ (indicating zero flow) and $Z_{ab} < 0$ (indicating a negative reduced cost, suggesting this arc could improve the solution). A solid red line represents arc (c,d), with labels $X_{cd} > 0$ (positive flow) and $Z_{cd} = 0$ (reduced cost is zero). The node r in tree I has a dual variable y=0. The notation suggests that the algorithm is about to select arc (a,b) as the entering arc because its reduced cost is negative. The context of the course (Linear Programming) is crucial for understanding the application of the Primal Network Simplex method to solve network flow optimization problems. The diagram visually represents the network structure and the associated flow and cost information before a simplex iteration.

Figures



(a) A diagram showing two green circles representing two trees in a network flow graph. The circles are labeled with Roman numerals I and II. The circles are connected by a dashed line and a solid red line. The dashed line is labeled with $X_{ab} = 0$ and $Z_{ab} < 0$. The solid red line is labeled with $X_{cd} > 0$ and $Z_{cd} = 0$. The nodes a, b, c, and d are labeled. There is a small vertical line segment at the bottom right of the right circle, labeled with r and y=0.

Slide 13

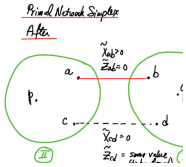
Content

Primal Network Simplex
After
[scale=0.8] [green!60!black, thick] (0,0) circle (1.5); [green!60!black, thick] (4,0) circle (1.5); at (0,0) p.; at (4,0) q.; [circle,fill,inner sep=1pt] (a) at (1.2,0.7) ; [circle,fill,inner sep=1pt] (b) at (2.8,0.7) ; [circle,fill,inner sep=1pt] (c) at (1.2,-0.7) ; [circle,fill,inner sep=1pt] (d) at (2.8,-0.7) ; [above] at (a) a; [above] at (b) b; [below] at (c) c; [below] at (d) d; [red] (a) – node[above]$\tilde{X}_{ab} \geq 0$ node[below]$\tilde{Z}_{ab} = 0$ (b); [dashed] (c) – node[below]$\tilde{X}_{cd} = 0$ node[above]$\tilde{Z}_{cd}$ = some value (d); at (0,-1.8) $\mathbb{III}$; at (4,-1.8) $\mathbb{I}$; [circle,fill,inner sep=1pt] (r) at (4.5, -0.7) ; [left] at (r) r; [above] at (4.5, -0.3) $\tilde{y} = 0$; (r) – (5,-0.7); (5,-0.7) – (5,-0.5); (5,-0.7) – (5,-0.9);

Description

Slide 13 of 45 in a Linear Programming course illustrates the state of a network flow graph *after* an iteration of the Primal Network Simplex method. The graph shows two trees (green circles labeled I and II) connected by arcs. The previous slide showed the situation before the iteration. Now, arc (a,b) has become part of the spanning tree (solid red line), with flow $\tilde{X}_{ab} \geq 0$ and reduced cost $\tilde{Z}_{ab} = 0$. Arc (c,d) is not in the spanning tree (dashed line), with flow $\tilde{X}_{cd} = 0$ and reduced cost $\tilde{Z}_{cd}$ which needs to be calculated. The tilde ( ) over the variables indicates that these are values after the iteration. Node r in tree I has a dual variable $\tilde{y} = 0$. The change from the previous slide reflects the update of the spanning tree and the dual variables during the simplex iteration. The context of the course (Linear Programming) is crucial for understanding the application of the Primal Network Simplex method to solve network flow optimization problems. The diagram visually represents the network structure and the associated flow and cost information after a simplex iteration.

Figures



(a) A diagram showing two green circles representing two trees in a network flow graph. The circles are labeled with Roman numerals I and II. The circles are connected by a solid red line and a dashed line. The solid red line is labeled with $\tilde{X}_{ab} \geq 0$ and $\tilde{Z}_{ab} = 0$. The dashed line is labeled with $\tilde{X}_{cd} = 0$ and $\tilde{Z}_{cd} =$ some value (to be found). The nodes a, b, c, and d are labeled. There is a small vertical line segment at the bottom right of the right circle, labeled with r and $\tilde{y} = 0$.

Slide 14

Content

Primal Network Simplex
Updating dual variables

In $\mathbb{I}$ (tree containing the root node):

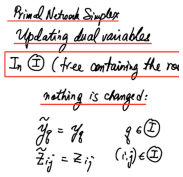
nothing is changed:

$\tilde{y}_i = y_i \quad g \in \mathbb{I}$
$\tilde{z}_{ij} = z_{ij} \quad (i,j) \in \mathbb{I}$

Description

Slide 14 of 45 in a Linear Programming course describes the updating of dual variables in the Primal Network Simplex method. Specifically, it focuses on the case where the update is performed on a tree (denoted by $\mathbb{I}$) containing the root node. The slide states that in this tree, nothing changes: the dual variables y_i remain unchanged ($\tilde{y}_i = y_i$), and the reduced costs z_{ij} also remain unchanged ($\tilde{z}_{ij} = z_{ij}$). The tilde ( ) denotes the updated values. The notation $\mathbb{I}$ represents a specific tree in the network graph, and the condition that it contains the root node is crucial for this step in the algorithm. The context of the course (Linear Programming) is essential for understanding the role of dual variables and reduced costs in the optimization process. The slide highlights a specific case within the broader Primal Network Simplex algorithm, showing how the dual variables are updated (or not updated in this case) during an iteration.

Figures



(a) There are no figures in this slide.

Slide 15

Content

Primal Network Simplex
Updating dual variables

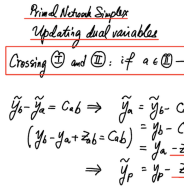
Crossing $\mathbb{I}$ and $\mathbb{III}$: if $a \in \mathbb{III} \longrightarrow b \in \mathbb{I}$

$_b - \tilde{y}_a = c_{ab} \implies \tilde{y}_a = \tilde{y}_b - c_{ab}$
$(\tilde{y}_b - \tilde{y}_a + \tilde{z}_{ab} = c_{ab}) = y_b - c_{ab}$
$= y_a - z_{ab}$
$\implies \tilde{y}_p = y_p - z_{ab}, \quad p \in \mathbb{III}$

Description

Slide 15 of 45 in a Linear Programming course details the update of dual variables in the Primal Network Simplex algorithm when an arc crosses between two trees in the network. The scenario is described as "Crossing $\mathbb{I}$ and $\mathbb{III}$," where $\mathbb{I}$ and $\mathbb{III}$ represent two disjoint trees in the network graph. An arc (a,b) is considered, with node a belonging to tree $\mathbb{III}$ and node b belonging to tree $\mathbb{I}$. The equation $\tilde{y}_b - \tilde{y}_a = c_{ab}$ shows the relationship between the dual variables ($\tilde{y}$) at nodes a and b and the cost c_{ab} of the arc (a,b). The tilde ( ) indicates updated values after the iteration. The equation is then manipulated to solve for $\tilde{y}_a$. The next line shows an alternative form of the equation, incorporating the reduced cost $\tilde{z}_{ab}$. Finally, the slide concludes that for any node p in tree $\mathbb{III}$, the updated dual variable $\tilde{y}_p$ is obtained by subtracting the reduced cost z_{ab} from the original dual variable y_p. The underlining highlights the key result of the update. The context of the course (Linear Programming) is crucial for understanding the role of dual variables and reduced costs in the network simplex method. This slide provides a specific formula for updating dual variables during a simplex iteration when arcs connect different trees.

Figures



(a) There are no figures in this slide.

Slide 16

Content

Primal Network Simplex
Updating dual variables

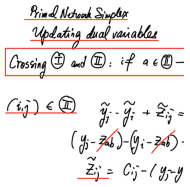
Crossing $\mathbb{I}$ and $\mathbb{III}$: if $a \in \mathbb{III} \longrightarrow b \in \mathbb{I}$

(i,j) $\in \mathbb{III}_j - \tilde{y}_i + \tilde{z}_{ij} = c_{ij}(\tilde{y}_j - z_{ab}) - (\tilde{y}_i - z_{ab}) + \tilde{z}_{ij} = c_{ij}\tilde{z}_{ij} = c_{ij} - (\tilde{y}_j - \tilde{y}_i) = z_{ij}$

Description

Slide 16 of 45 in a Linear Programming course continues the explanation of updating dual variables in the Primal Network Simplex method. Building upon slide 15, this slide focuses on the calculation of the updated reduced cost $\tilde{z}_{ij}$ for an arc (i, j) that belongs to tree $\mathbb{III}$. The equation $\tilde{y}_j - \tilde{y}_i + \tilde{z}_{ij} = c_{ij}$ relates the updated dual variables $\tilde{y}_j$ and $\tilde{y}_i$ at nodes j and i, the updated reduced cost $\tilde{z}_{ij}$, and the original cost c_{ij} of the arc. The next line shows a manipulation of the equation, seemingly attempting to incorporate the reduced cost z_{ab} from the previous step (crossing arc (a,b)). However, the subsequent line simplifies this to $\tilde{z}_{ij} = c_{ij} - (\tilde{y}_j - \tilde{y}_i) = z_{ij}$, indicating that the updated reduced cost $\tilde{z}_{ij}$ is equal to the original reduced cost z_{ij}. The underlining highlights the key equations and results. The tilde ( ) denotes updated values. The Roman numerals $\mathbb{I}$ and $\mathbb{III}$ represent distinct trees in the network graph. The context of the course (Linear Programming) is essential for understanding the algorithm's steps and the significance of dual variables and reduced costs in network optimization problems. This slide provides a specific formula for updating reduced costs during a simplex iteration when arcs are within a specific tree.

Figures



(a) There are no figures in this slide.

Slide 17

Content

Primal Network Simplex
Updating dual variables

Crossing $\mathbb{I}$ and $\mathbb{III}$: if $a \in \mathbb{III} \longrightarrow b \in \mathbb{I}$

$i \in \mathbb{III}, \quad j \in \mathbb{I}$

$\tilde{y}_j - \tilde{y}_i + \tilde{z}_{ij} = c_{ij}$

$\tilde{y}_j - (\tilde{y}_i - z_{ab}) + \tilde{z}_{ij} = c_{ij}$

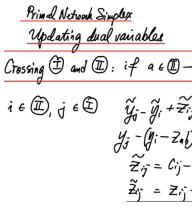
$\tilde{z}_{ij} = c_{ij} - (\tilde{y}_j - \tilde{y}_i) - z_{ab}$

$\tilde{z}_{ij} = z_{ij} - z_{ab}$

Description

Slide 17 of 45 in a Linear Programming course continues the discussion on updating dual variables within the Primal Network Simplex algorithm. This slide focuses on the case where an arc connects two different trees, $\mathbb{I}$ and $\mathbb{III}$, in the network. The notation $a \in \mathbb{III} \longrightarrow b \in \mathbb{I}$ indicates an arc from node a in tree $\mathbb{III}$ to node b in tree $\mathbb{I}$. The slide then considers an arc (i, j) where $i \in \mathbb{III}$ and $j \in \mathbb{I}$. The equation $\tilde{y}_j - \tilde{y}_i + \tilde{z}_{ij} = c_{ij}$ shows the relationship between updated dual variables $(\tilde{y}_j, \tilde{y}_i)$ at nodes j and i, the updated reduced cost $\tilde{z}_{ij}$, and the original cost c_{ij} of the arc. The next line substitutes the updated dual variable $\tilde{y}_i$ with an expression involving the reduced cost z_{ab} from the crossing arc (a,b) as derived in the previous slide. The subsequent lines manipulate this equation to solve for the updated reduced cost $\tilde{z}_{ij}$, resulting in $\tilde{z}_{ij} = c_{ij} - (\tilde{y}_j - \tilde{y}_i) - z_{ab}$. Finally, this is simplified to $\tilde{z}_{ij} = z_{ij} - z_{ab}$, showing that the updated reduced cost is obtained by subtracting z_{ab} from the original reduced cost z_{ij}. The underlining highlights the final result. The tilde ( ) denotes updated values. The context of the course (Linear Programming) is crucial for understanding the algorithm's steps and the significance of dual variables and reduced costs in network optimization problems. This slide provides a specific formula for updating reduced costs during a simplex iteration when arcs connect different trees, building upon the previous slide's derivation.

Figures



(a) There are no figures in this slide.

Slide 18

Content

Primal Network Simplex
Updating dual variables

Crossing $\mathbb{I}$ and $\mathbb{III}$: if $a \in \mathbb{III} \longrightarrow b \in \mathbb{I}$

$i \in \mathbb{I}, \quad j \in \mathbb{III}$

$\tilde{y}_j - \tilde{y}_i + \tilde{z}_{ij} = c_{ij}$

$(\tilde{y}_j - z_{ab}) - \tilde{y}_i + \tilde{z}_{ij} = c_{ij}$

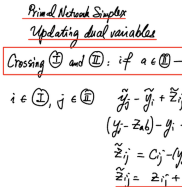
$\tilde{z}_{ij} = c_{ij} - (\tilde{y}_j - \tilde{y}_i) + z_{ab}$

$\tilde{z}_{ij} = z_{ij} + z_{ab}$

Description

Slide 18 of 45 in a Linear Programming course continues the discussion on updating dual variables in the Primal Network Simplex algorithm. This slide focuses on updating the reduced cost $\tilde{z}_{ij}$ for an arc (i, j) where node i belongs to tree $\mathbb{I}$ and node j belongs to tree $\mathbb{III}$. The notation $a \in \mathbb{III} \longrightarrow b \in \mathbb{I}$ indicates that a previous iteration involved an arc from node a in tree $\mathbb{III}$ to node b in tree $\mathbb{I}$. The equation $\tilde{y}_j - \tilde{y}_i + \tilde{z}_{ij} = c_{ij}$ relates the updated dual variables $\tilde{y}_j$ and $\tilde{y}_i$ at nodes j and i, the updated reduced cost $\tilde{z}_{ij}$, and the original cost c_{ij} of the arc. The next line substitutes the updated dual variable $\tilde{y}_j$ with an expression involving the reduced cost z_{ab} from the crossing arc (a,b) as derived in the previous slide. The subsequent lines manipulate this equation to solve for the updated reduced cost $\tilde{z}_{ij}$, resulting in $\tilde{z}_{ij} = c_{ij} - (\tilde{y}_j - \tilde{y}_i) + z_{ab}$. Finally, this is simplified to $\tilde{z}_{ij} = z_{ij} + z_{ab}$, showing that the updated reduced cost is obtained by adding z_{ab} to the original reduced cost z_{ij}. The underlining highlights the final result. The tilde ( ) denotes updated values. The context of the course (Linear Programming) is crucial for understanding the algorithm's steps and the significance of dual variables and reduced costs in network optimization problems. This slide provides a specific formula for updating reduced costs during a simplex iteration when arcs connect different trees, extending the derivation from the previous slides.

Figures



(a) There are no figures in this slide.

Slide 19

Content

Primal Network Simplex
Updating dual variables

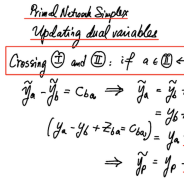
Crossing $\mathbb{I}$ and $\mathbb{III}$: if $a \in \mathbb{III} \longleftarrow b \in \mathbb{I}$

$_a - \tilde{y}_b = c_{ba} \implies \tilde{y}_a = \tilde{y}_b + c_{ba}$
$= y_b + c_{ba}$
$(y_a - y_b + z_{ba} = c_{ba}) = y_a + z_{ba}$
$\implies \tilde{y}_p = y_p + z_{ba}, p \in \mathbb{I}$

Description

Slide 19 of 45 in a Linear Programming course describes the updating of dual variables in the Primal Network Simplex method. The slide focuses on the scenario where an arc crosses between two trees, denoted as $\mathbb{I}$ and $\mathbb{III}$ in the network. The notation $a \in \mathbb{III} \longleftarrow b \in \mathbb{I}$ indicates an arc from node b in tree $\mathbb{I}$ to node a in tree $\mathbb{III}$. The equation $\tilde{y}_a - \tilde{y}_b = c_{ba}$ shows the relationship between the updated dual variables $\tilde{y}_a$ and $\tilde{y}_b$ at nodes a and b, and the cost c_{ba} of the arc. The subsequent lines manipulate this equation, incorporating the reduced cost z_{ba} associated with the arc (b,a). The final line shows that the updated dual variable $\tilde{y}_p$ for any node p in tree $\mathbb{I}$ is obtained by adding the reduced cost z_{ba} to the original dual variable y_p. The tilde ( ) denotes updated values. The underlining highlights key equations and results. The context of the course (Linear Programming) is essential for understanding the algorithm's steps and the significance of dual variables and reduced costs in network optimization problems. This slide provides a specific formula for updating dual variables during a simplex iteration when an arc connects nodes in different trees.

Figures



(a) There are no figures in this slide.

Slide 20

Content

Primal Network Simplex
Updating dual variables

Crossing $\mathbb{I}$ and $\mathbb{III}$: if $a \in \mathbb{III} \longleftarrow b \in \mathbb{I}$

(ij) $\in \mathbb{I}$

$\tilde{y}_j - \tilde{y}_i + \tilde{z}_{ij} = c_{ij}$

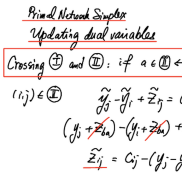
$(\tilde{y}_j + z_{ba}) - (\tilde{y}_i + z_{ba}) + \tilde{z}_{ij} = c_{ij}$

$\tilde{z}_{ij} = c_{ij} - (y_j - y_i) = z_{ij}$

Description

Slide 20 of 45 in a Linear Programming course focuses on updating dual variables within the Primal Network Simplex algorithm. The slide specifically addresses the situation where an arc connects nodes within the same tree ($\mathbb{I}$) while a previous iteration involved an arc crossing between trees $\mathbb{I}$ and $\mathbb{III}$. The notation $a \in \mathbb{III} \longleftarrow b \in \mathbb{I}$ indicates an arc from node b in tree $\mathbb{I}$ to node a in tree $\mathbb{III}$. The equation $\tilde{y}_j - \tilde{y}_i + \tilde{z}_{ij} = c_{ij}$ shows the relationship between updated dual variables ($\tilde{y}_j$, $\tilde{y}_i$) at nodes j and i, the updated reduced cost $\tilde{z}_{ij}$, and the original cost c_{ij} of the arc (i, j) within tree $\mathbb{I}$. The next line shows an attempt to incorporate the effect of the crossing arc (b,a) with reduced cost z_{ba} on the updated dual variables. However, there's a mistake in the handwritten notes; the z_{ba} terms cancel out incorrectly. The final line incorrectly concludes that the updated reduced cost $\tilde{z}_{ij}$ is equal to the original reduced cost z_{ij}. The underlining highlights the incorrect final result. The tilde ( ) denotes updated values. The context of the course (Linear Programming) is crucial for understanding the algorithm's steps and the significance of dual variables and reduced costs in network optimization problems. This slide demonstrates an error in the derivation of the update rule for reduced costs when arcs are within the same tree, but a crossing arc has been previously considered. The error lies in the incorrect cancellation of the z_{ba} terms.

Figures



(a) There are no figures in this slide.

Slide 21

Content

Primal Network Simplex
Updating dual variables

Crossing $\mathbb{I}$ and $\mathbb{III}$: if $a \in \mathbb{III} \longleftarrow b \in \mathbb{I}$

$i \in \mathbb{III}, \quad j \in \mathbb{I}$

$\tilde{y}_j - \tilde{y}_i + \tilde{z}_{ij} = c_{ij}$

$\tilde{y}_j - (\tilde{y}_i + z_{ba}) + \tilde{z}_{ij} = c_{ij}$

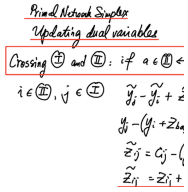
$\tilde{z}_{ij} = c_{ij} - (\tilde{y}_j - \tilde{y}_i) + z_{ab}$

$\tilde{z}_{ij} = z_{ij} + z_{ab}$

Description

Slide 21 of 45 in a Linear Programming course continues the discussion on updating dual variables in the Primal Network Simplex algorithm. This slide focuses on updating the reduced cost $\tilde{z}_{ij}$ for an arc (i, j) where node i belongs to tree $\mathbb{III}$ and node j belongs to tree $\mathbb{I}$. The notation $a \in \mathbb{III} \longleftarrow b \in \mathbb{I}$ indicates that a previous iteration involved an arc from node b in tree $\mathbb{I}$ to node a in tree $\mathbb{III}$. The equation $\tilde{y}_j - \tilde{y}_i + \tilde{z}_{ij} = c_{ij}$ relates the updated dual variables $\tilde{y}_j$ and $\tilde{y}_i$ at nodes j and i, the updated reduced cost $\tilde{z}_{ij}$, and the original cost c_{ij} of the arc. The next line substitutes the updated dual variable $\tilde{y}_i$ with an expression involving the reduced cost z_{ba} from the crossing arc (b,a) as derived in the previous slide. The subsequent lines manipulate this equation to solve for the updated reduced cost $\tilde{z}_{ij}$, resulting in $\tilde{z}_{ij} = c_{ij} - (\tilde{y}_j - \tilde{y}_i) + z_{ab}$. There is a slight inconsistency in the notation between z_{ab} and z_{ba} which may be a typographical error. Finally, this is simplified to $\tilde{z}_{ij} = z_{ij} + z_{ab}$, showing that the updated reduced cost is obtained by adding z_{ab} to the original reduced cost z_{ij}. The underlining highlights the final result. The tilde ( ) denotes updated values. The context of the course (Linear Programming) is crucial for understanding the algorithm's steps and the significance of dual variables and reduced costs in network optimization problems. This slide provides a specific formula for updating reduced costs during a simplex iteration when arcs connect different trees, extending the derivation from the previous slides. There is a potential minor error in the derivation, as the final equation seems to assume that the change in dual variables for node i is solely due to the crossing arc, which might not always be the case.

Figures



(a) There are no figures in this slide.

Slide 22

Content

Primal Network Simplex
Updating dual variables

Crossing $\mathbb{I}$ and $\mathbb{III}$: if $a \in \mathbb{III} \longleftarrow b \in \mathbb{I}$

$i \in \mathbb{I}, \quad j \in \mathbb{III}$

$\tilde{y}_j - \tilde{y}_i + \tilde{z}_{ij} = c_{ij}$

$(y_j + z_{ba}) - y_i + \tilde{z}_{ij} = c_{ij}$

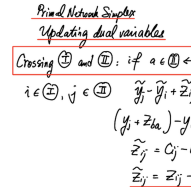
$\tilde{z}_{ij} = c_{ij} - (y_j - y_i) - z_{ab}$

$\tilde{z}_{ij} = z_{ij} - z_{ab}$

Description

Slide 22 of 45 in a Linear Programming course focuses on updating dual variables and reduced costs in the Primal Network Simplex method. The slide addresses the scenario where an arc connects nodes in different trees, $\mathbb{I}$ and $\mathbb{III}$, following a previous iteration where an arc crossed between these trees. The notation $a \in \mathbb{III} \longleftarrow b \in \mathbb{I}$ indicates an arc from node b in tree $\mathbb{I}$ to node a in tree $\mathbb{III}$. The equation $\tilde{y}_j - \tilde{y}_i + \tilde{z}_{ij} = c_{ij}$ shows the relationship between updated dual variables $\tilde{y}_j$ and $\tilde{y}_i$, the updated reduced cost $\tilde{z}_{ij}$, and the original cost c_{ij} of the arc (i, j). The next line substitutes the updated dual variable $\tilde{y}_j$ with an expression incorporating the reduced cost z_{ba} from the crossing arc (b,a), reflecting the update from the previous slide. The subsequent lines manipulate this equation to solve for the updated reduced cost $\tilde{z}_{ij}$. The final line presents the result $\tilde{z}_{ij} = z_{ij} - z_{ab}$, indicating that the updated reduced cost is obtained by subtracting z_{ab} from the original reduced cost z_{ij}. There's a potential inconsistency in the notation between z_{ab} and z_{ba} which might be a typographical error. The underlining highlights the final result. The tilde ( ) denotes updated values. The context of the course (Linear Programming) is crucial for understanding the algorithm's steps and the significance of dual variables and reduced costs in network optimization problems. This slide provides a specific formula for updating reduced costs when arcs connect nodes in different trees, building upon the derivations from previous slides. The derivation assumes that the change in the dual variable for node j is solely due to the crossing arc, which might not always be true, potentially leading to an error in the final equation.

Figures



(a) There are no figures in this slide.

Slide 23

Content

Primal Network Simplex
Updating dual variables
Dual variables update:
- If the entering arc crosses from the root-containing tree to the non-root-containing tree, then increase all dual variables on the non-root-containing tree by the dual slack of the entering arc.
- Otherwise, decrease these dual variables by this amount.

Dual slacks update:
- The dual slacks corresponding to those arcs that bridge in the same direction as the entering arc get decremented by the old dual slack on the entering arc, whereas those that correspond to arcs bridging in the opposite direction get incremented by this amount.

Description

Slide 23 of 45 in a Linear Programming course describes the update rules for dual variables and dual slacks in the Primal Network Simplex algorithm. The slide is divided into two sections: "Dual variables update" and "Dual slacks update".

The "Dual variables update" section outlines two cases:

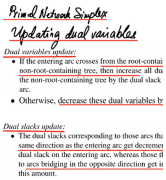
1. If the entering arc connects the root-containing tree to a non-root-containing tree, then all dual variables in the non-root-containing tree are increased by the dual slack of the entering arc. 2. Otherwise (if the entering arc connects nodes within the same tree), these dual variables are decreased by the dual slack of the entering arc.

The "Dual slacks update" section explains how the dual slacks of the arcs are updated:

Dual slacks corresponding to arcs bridging in the same direction as the entering arc are decremented by the old dual slack on the entering arc. Dual slacks corresponding to arcs bridging in the opposite direction are incremented by the old dual slack on the entering arc.

The context of Linear Programming is crucial here, as these update rules are specific to the network simplex method used to solve linear programs represented as networks. The terms "dual variables," "dual slacks," "root-containing tree," and "non-root-containing tree" are all standard terminology within network optimization algorithms. The slide provides a concise summary of the update rules, crucial for implementing the Primal Network Simplex algorithm.

Figures



(a) There are no figures in this slide.

Slide 24

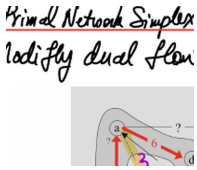
Content

Primal Network Simplex
Modifly dual flow

Description

Slide 24 of 45 in a Linear Programming course illustrates the modification of dual flows in the Primal Network Simplex method. The slide features Figure 14.8, a network graph depicting two disjoint trees, labeled $\mathbb{I}$ and $\mathbb{III}$. The nodes are labeled a through g. Arcs between nodes represent edges in the network, with numerical values indicating either primal values (flows) or dual values (costs or potentials). Some arcs have question marks, indicating that the dual values associated with those arcs need to be updated. The red arrows highlight the entering arc (with a flow of 3) and the leaving arc (with a flow of 9). The caption explains that the figure shows two disjoint trees, with unchanged primal and dual values shown, while those requiring updates are marked with question marks. The context of the course (Linear Programming) is crucial for understanding the network simplex algorithm and the role of dual variables in finding optimal solutions. The figure visually represents the state of the network before the update step in the algorithm, highlighting the arcs that will be affected by the change in flow.

Figures



(a) Figure 14.8 showing two disjoint trees. Primal and dual values that remained unchanged are shown, whereas those that need to be updated are shown as question marks.

Slide 25

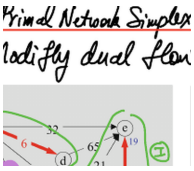
Content

Primal Network Simplex
Modifly dual flow
50 = 41 - (-9)
78 = 69 - (-9)

Description

Slide 25 of 45 in a Linear Programming course demonstrates the modification of dual flows in the Primal Network Simplex method. The slide presents a before-and-after comparison of a network graph, showing two disjoint trees ($\mathbb{I}$ and $\mathbb{III}$) before and after updating dual flows. The left side shows the initial state, with dual values (costs or potentials) on the arcs. The right side shows the updated state after an iteration of the algorithm. The entering arc (flow 3) and leaving arc (flow 9) are implicitly identified by the change in flow values. The calculations $50 = 41 - (-9)$ and $78 = 69 - (-9)$ illustrate how the dual values are updated based on the change in flow and the dual slack of the entering arc (-9). The updated dual values are shown on the right-hand side of the figure. Some dual values remain unchanged, as indicated by the "no change" annotation. The nodes are labeled a through g, and the arcs are labeled with their flows and dual values. The context of the course (Linear Programming) is crucial for understanding the network simplex algorithm and the role of dual variables in finding optimal solutions. The figure visually represents the update step in the algorithm, highlighting the changes in dual flows resulting from the change in primal flows.

Figures



(a) Figure showing two disjoint trees before and after updating dual flows. The left tree shows the initial state, with dual values on the arcs. The right tree shows the updated state after modifying the dual flow, with some dual values updated and others remaining unchanged. The entering arc has a flow of 3, and the leaving arc has a flow of 9. The updated dual values are calculated using the formulas shown above the figure. The arcs are labeled with their flows and dual values. Nodes are labeled a through g. The trees are labeled $\mathbb{I}$ and $\mathbb{III}$.

Slide 26

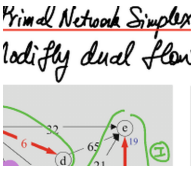
Content

Primal Network Simplex
Modifly dual flow
41 = 32 - (-9)
18 = 9 - (-9)
-10 = -1 + (-9)

Description

Slide 26 of 45 in a Linear Programming course visually demonstrates the update of dual flows in the Primal Network Simplex method. It shows two network graphs representing two disjoint trees, labeled $\mathbb{I}$ and $\mathbb{III}$, before and after the update. The left graph shows the initial state, with dual values (costs or potentials) annotated on the arcs. The right graph shows the updated state after modifying the dual flow. The changes are highlighted by the updated dual values on some arcs. The calculations $41 = 32 - (-9)$, $18 = 9 - (-9)$, and $-10 = -1 + (-9)$ illustrate how the dual values are updated based on the change in flow and the dual slack of the entering arc (-9). The entering arc and leaving arc are implicitly identified by the change in flow values. The nodes are labeled a through g, and the arcs are labeled with their flows and dual values. The context of the course (Linear Programming) is crucial for understanding the network simplex algorithm and the role of dual variables in finding optimal solutions. The figure visually represents the update step in the algorithm, highlighting the changes in dual flows resulting from the change in primal flows. The equations show the specific calculations used to update the dual values on selected arcs.

Figures



(a) Figure showing two disjoint trees before and after updating dual flows. The left tree shows the initial state, with dual values on the arcs. The right tree shows the updated state after modifying the dual flow, with some dual values updated and others remaining unchanged. The entering arc has a flow of 3, and the leaving arc has a flow of 9. The updated dual values are calculated using the formulas shown above the figure. The arcs are labeled with their flows and dual values. Nodes are labeled a through g. The trees are labeled $\mathbb{I}$ and $\mathbb{III}$.

Slide 27

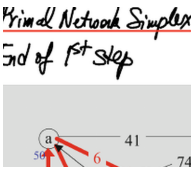
Content

Primal Network Simplex
End of 1st step

Description

Slide 27 of 45 in a Linear Programming course shows the network flow graph at the end of the first step of the Primal Network Simplex algorithm. The graph consists of seven nodes (a, b, c, d, e, f, g) connected by arcs representing the flow of a commodity. Each arc is labeled with two numbers: the flow value (in black) and the dual value (in blue). The nodes also have values (in blue) representing node potentials. Red arrows indicate the direction of flow. The graph shows two disjoint spanning trees. The context of the course (Linear Programming) is crucial for understanding the network simplex algorithm and its application to solving optimization problems. The figure visually represents the intermediate solution obtained after the first iteration of the algorithm, highlighting the current flow values and dual variables.

Figures



(a) A network flow graph showing nodes a, b, c, d, e, f, g. Arcs between nodes are labeled with flow values (in black) and dual values (in blue). Node values are shown in blue. Red arrows indicate the direction of flow along certain arcs. The flows are 3, 6, 9, 2, 4, 3, 2, 18, 41, 74, 21, 9, 3. The dual values are 50, 33, 98, 78, 19, -6, 0, -10, 41, 74, 21, 18. The graph shows two disjoint trees.

Slide 28

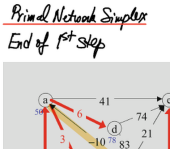
Content

Primal Network Simplex
End of 1st step

Description

Slide 28 of 45 in a Linear Programming course displays the network flow graph at the end of the first step of the Primal Network Simplex algorithm. The main part of the slide shows a network graph with seven nodes (a, b, c, d, e, f, g) and arcs representing flow. Each arc is labeled with its flow (black) and dual value (blue). Node potentials are also shown in blue. Red arrows indicate flow direction. The graph shows two disjoint spanning trees. A smaller graph on the right illustrates a portion of the network, highlighting flow adjustments during the algorithm's iteration. Labels "enter" and "leave" indicate the entering and leaving arcs, respectively. The variable t represents the change in flow along the entering arc, with $t = 3$ indicating the step size. The context of the course (Linear Programming) is crucial for understanding the network simplex algorithm and its application to optimization problems. The figure visually represents the intermediate solution after the first iteration, showing current flow values and dual variables. The smaller graph provides a zoomed-in view of the flow adjustments made during this iteration.

Figures



(a) A network flow graph showing nodes a, b, c, d, e, f, g. Arcs between nodes are labeled with flow values (in black) and dual values (in blue). Node values are shown in blue. Red arrows indicate the direction of flow along certain arcs. The flows are 3, 6, 9, 2, 4, 3, 2, 18, 41, 74, 21, 9, 3. The dual values are 50, 33, 98, 78, 19, -6, 0, -10, 41, 74, 21, 18. The graph shows two disjoint trees. A smaller graph to the right shows a portion of the network with flow adjustments indicated by arrows and labels, including "enter", "leave", and "t=3".

Slide 29

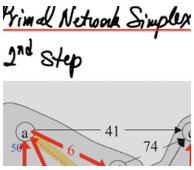
Content

Primal Network Simplex
2nd Step

Description

Slide 29 of 45 in a Linear Programming course shows the network flow graph during the second step of the Primal Network Simplex algorithm. The main graph displays a network with seven nodes (a, b, c, d, e, f, g) and arcs representing flow. Each arc is labeled with its flow (black) and dual value (blue). Node potentials are shown in blue. Red arrows indicate flow direction. Some arcs are highlighted in yellow/orange to indicate the path of flow adjustment. The graph shows two disjoint spanning trees. A smaller graph on the right illustrates a portion of the network, highlighting flow adjustments during this iteration. Labels "enter" and "leave" indicate the entering and leaving arcs, respectively. The variable t represents the change in flow along the entering arc, with $t = 3$ indicating the step size. The smaller graph depicts a cycle with flow adjustments, illustrating how the flow is updated along the cycle. The context of the course (Linear Programming) is crucial for understanding the network simplex algorithm and its application to optimization problems. The figure visually represents the intermediate solution after the second iteration, showing current flow values and dual variables. The smaller graph provides a zoomed-in view of the flow adjustments made during this iteration, specifically highlighting the cycle used for flow update.

Figures



(a) A network flow graph showing nodes a, b, c, d, e, f, g. Arcs between nodes are labeled with flow values (in black) and dual values (in blue). Node values are shown in blue. Red arrows indicate the direction of flow along certain arcs. Some arcs are highlighted in yellow/orange. The flows are 6, 3, 9, 4, 2, 3, 2, 18, 41, 74, 21, 9, 3. The dual values are 50, 33, 98, 78, 19, -6, 0, -10, 41, 74, 21, 18. The graph shows two disjoint trees. A smaller graph to the right shows a portion of the network with flow adjustments indicated by arrows and labels, including "enter", "leave", and "t=3". The smaller graph shows a triangle with flows 3+t, 3-t, and 3, with t=3.

Slide 30

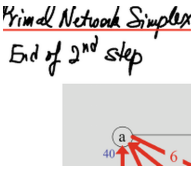
Content

Primal Network Simplex
End of 2nd step

Description

Slide 30 of 45 in a Linear Programming course shows the network flow graph at the end of the second step of the Primal Network Simplex algorithm. The graph consists of seven nodes (a, b, c, d, e, f, g) connected by arcs representing the flow of a commodity. Each arc is labeled with two numbers: the flow value (in black) and the dual value (in blue). The nodes also have values (in blue) representing node potentials. Red arrows indicate the direction of flow. The graph shows a connected network, unlike the previous slide which showed two disjoint trees. The values on the arcs and nodes represent the current solution after two iterations of the algorithm. The context of the course (Linear Programming) is crucial for understanding the network simplex algorithm and its application to solving optimization problems. The figure visually represents the intermediate solution obtained after the second iteration of the algorithm, highlighting the current flow values and dual variables. The change from two disjoint trees to a single connected network indicates progress towards an optimal solution.

Figures



(a) A network flow graph showing nodes a, b, c, d, e, f, g. Arcs between nodes are labeled with flow values (in black) and dual values (in blue). Node values are shown in blue. Red arrows indicate the direction of flow along certain arcs. The flows are 6, 6, 9, 3, 2, 4, 8, 31, 64, 21, 10, 2, 3. The dual values are 40, 33, 88, 73, 19, -16, 0, 6, 64, 21, 10, 2, 3. The graph shows a connected network.

Slide 31

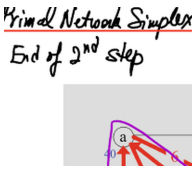
Content

Primal Network Simplex
End of 2nd step

Description

Slide 31 of 45 in a Linear Programming course displays the network flow graph at the end of the second step of the Primal Network Simplex algorithm. The graph shows seven nodes (a, b, c, d, e, f, g) connected by arcs representing flow. Each arc is labeled with its flow (black) and dual value (blue). Node potentials are shown in blue. Red arrows indicate flow direction. A purple loop highlights a cycle indicating a flow adjustment. The values on the arcs and nodes represent the current solution after two iterations of the algorithm. A small circle to the left of the main graph shows "t=9", indicating the amount of flow adjustment along the cycle. The arc from f to a is labeled 9-t, and the arc from c to a is labeled 3+t, showing the flow changes. The graph shows a connected network, unlike the previous slide which showed two disjoint trees. The context of the course (Linear Programming) is crucial for understanding the network simplex algorithm and its application to solving optimization problems. The figure visually represents the intermediate solution obtained after the second iteration of the algorithm, highlighting the current flow values and dual variables. The purple cycle and the value of t highlight the flow adjustments made during this iteration.

Figures



(a) A network flow graph showing nodes a, b, c, d, e, f, g. Arcs between nodes are labeled with flow values (in black) and dual values (in blue). Node values are shown in blue. Red arrows indicate the direction of flow along certain arcs. The flows are 6, 10, 9, 3, 2, 4, 8, 31, 64, 21, 2, 3. The dual values are 40, 33, 88, 68, 19, -16, 0, 6, 64, 21, 2, 3. A purple loop highlights a cycle in the graph. The node values are 40, 33, 88, 68, 19, -16, 0. A small circle to the left shows "t=9". The arc from f to a has a flow of 9-t. The arc from c to a has a flow of 3+t. The arc from c to b has a flow of 10. The arc from f to b has a flow of 4. The arc from b to g has a flow of 3. The arc from f to a has a flow of 9. The arc from c to a has a flow of 6. The arc from a to d has a flow of 6. The arc from d to b has a flow of 3. The arc from d to e has a flow of 64. The arc from e to g has a flow of 2. The arc from b to f has a flow of 8. The arc from f to a has a flow of 9. The arc from c to a has a flow of 6. The arc from a to d has a flow of 6. The arc from d to b has a flow of 3. The arc from d to e has a flow of 64. The arc from e to g has a flow of 2. The arc from b to f has a flow of

Slide 32

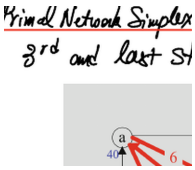
Content

Primal Network Simplex
3rd and last step: optimal

Description

Slide 32 of 45 in a Linear Programming course shows the network flow graph at the end of the third and final step of the Primal Network Simplex algorithm. The graph displays a network with seven nodes (a, b, c, d, e, f, g) and arcs representing flow. Each arc is labeled with its flow (black) and dual value (blue). Node potentials are shown in blue. Red arrows indicate flow direction. The graph shows a connected network. The values on the arcs and nodes represent the optimal solution found after three iterations of the algorithm. The title indicates that this is the final, optimal solution. The context of the course (Linear Programming) is crucial for understanding the network simplex algorithm and its application to solving optimization problems. The figure visually represents the optimal solution obtained after the third iteration of the algorithm, highlighting the final flow values and dual variables. The connected nature of the network indicates that a feasible solution has been found. The absence of any further flow adjustments suggests that the algorithm has converged to the optimal solution.

Figures



(a) A network flow graph showing nodes a, b, c, d, e, f, g. Arcs between nodes are labeled with flow values (in black) and dual values (in blue). Node values are shown in blue. Red arrows indicate the direction of flow along certain arcs. The flows are 6, 6, 10, 12, 2, 5, 9, 31, 64, 21, 3. The dual values are 40, 33, 88, 73, 19, -15, 0. The graph shows a connected network. The node values are 40, 33, 88, 73, 19, -15, 0.

Slide 33

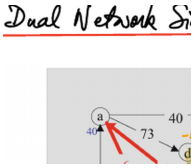
Content

Dual Network Simplex

Description

Slide 33 of 45 in a Linear Programming course presents a network flow graph representing a tree solution within the context of the Dual Network Simplex method. The graph shows seven nodes (a, b, c, d, e, f, g) connected by arcs. Each arc is labeled with a flow value (in black) and a dual value (in blue). Node potentials are also shown in blue. Red arrows indicate the direction of flow. The caption "FIGURE 14.13. A tree solution that is dual feasible but not primal feasible" indicates that the current solution satisfies the dual feasibility conditions but violates the primal feasibility conditions. This is a key concept in the Dual Network Simplex algorithm, where the algorithm iteratively improves a dual feasible solution until primal feasibility is also achieved. The graph visually represents an intermediate solution in the algorithm's execution. The context of the course (Linear Programming) is crucial for understanding the network simplex algorithm and its application to solving optimization problems. The figure visually represents an intermediate solution obtained during the algorithm's execution, highlighting the current flow values and dual variables. The tree structure of the network is characteristic of intermediate solutions in network simplex methods.

Figures



(a) A network flow graph showing nodes a, b, c, d, e, f, g. Arcs between nodes are labeled with flow values (in black) and dual values (in blue). Node values are shown in blue. Red arrows indicate the direction of flow along certain arcs. The flows are 6, 6, 10, 8, 2, 9, 5, 5. The dual values are 40, 33, 88, 73, 19, -15, 0. The graph shows a tree structure. The node values are 40, 33, 88, 73, 19, -15, 0. The arc from a to d has a flow of 6 and a dual value of 73. The arc from d to b has a flow of 8 and a dual value of -5. The arc from b to g has a flow of 5 and a dual value of 5. The arc from e to g has a flow of 5 and a dual value of 5. The arc from e to d has a flow of 2 and a dual value of 2. The arc from a to e has a flow of 2 and a dual value of -2. The arc from a to c has a flow of 6 and a dual value of 6. The arc from c to b has a flow of 10 and a dual value of 10. The arc from c to f has a flow of 5 and a dual value of 5. The arc from f to g has a flow of 9 and a dual value of 9. The arc from f to a has a flow of 1 and a dual value of 1. The arc from b to d has a flow of 6 and a dual value of -6. The arc from d to a has a flow of 6 and a dual value of -6. The arc from g to b has a flow of 5 and a dual value of

Slide 34

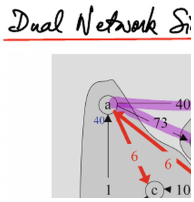
Content

Dual Network Simplex

Description

Slide 34 of 45 in a Linear Programming course illustrates the first pivot step in the Dual Network Simplex method. The figure shows a network flow graph with seven nodes (a, b, c, d, e, f, g) and arcs representing flow. Each arc is labeled with its flow (black) and dual value (blue). Node potentials are shown in blue. Red arrows indicate flow direction. The graph is divided into two subtrees, highlighted by the purple and red arcs. The text "leaving arc II to I" indicates that an arc from subtree II is being moved to subtree I during this pivot operation. The dual values on the arcs and nodes represent the current dual feasible solution. The figure visually represents the intermediate solution before the pivot, showing the two subtrees involved in the pivot operation. The context of the course (Linear Programming) is crucial for understanding the dual network simplex algorithm and its application to solving optimization problems. The figure visually represents the intermediate solution before the pivot, highlighting the current flow values and dual variables. The division into two subtrees is a key characteristic of the dual network simplex method.

Figures



(a) A network flow graph showing nodes a, b, c, d, e, f, g. Arcs between nodes are labeled with flow values (in black) and dual values (in blue). Node values are shown in blue. Red arrows indicate the direction of flow along certain arcs. The nodes are labeled a, b, c, d, e, f, g. The node values are 40, 33, 88, 5, 10, -15, 0. The arc values are shown in black and blue. A purple arc connects nodes a and e. A purple arc connects nodes d and e. A red arc connects nodes a and c. A red arc connects nodes a and d. A red arc connects nodes c and b. A red arc connects nodes b and g. A red arc connects nodes f and b. A red arc connects nodes f and a. The text "leaving arc II to I" is written to the right of the graph. The graph shows two subtrees. The figure is labeled "FIGURE 14.14. The two subtrees for the first pivot of the dual simplex method."

Slide 35

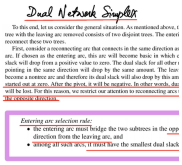
Content

Dual Network Simplex
To this end, let us consider the general situation. As mentioned above, the spanning tree with the leaving arc removed consists of two disjoint trees. The entering arc must reconnect these two trees.
First, consider a reconnecting arc that connects in the same direction as the leaving arc. If chosen as the entering arc, this arc will become basic in which case its dual slack will drop from a positive value to zero. The dual slack for all other nontree arcs pointing in the same direction will drop by the same amount. The leaving arc will become a nontree arc and therefore its dual slack will also drop by this amount. But, it started out at zero. After the pivot, it will be negative. In other words, dual feasibility will be lost. For this reason, we restrict our attention to reconnecting arcs that point in the opposite direction.
Entering arc selection rule:
• the entering arc must bridge the two subtrees in the opposite direction from the leaving arc, and
• among all such arcs, it must have the smallest dual slack.

Description

Slide 35 of 45 in a Linear Programming course details the Dual Network Simplex method, specifically addressing the selection rule for the entering arc. The slide explains that removing the leaving arc from the spanning tree results in two disjoint subtrees. The entering arc must reconnect these subtrees. Crucially, the entering arc must connect the subtrees in the opposite direction of the leaving arc to maintain dual feasibility. If the entering arc were chosen in the same direction, the dual slack of the leaving arc would become negative after the pivot, violating dual feasibility. The selection rule prioritizes the arc with the smallest dual slack among those satisfying the directional constraint. The underlined phrases emphasize the importance of the opposite direction and the smallest dual slack for maintaining feasibility and efficiency in the algorithm. The context of linear programming is essential because this slide describes a crucial step in a network optimization algorithm used to solve linear programs.

Figures



(a) Handwritten notes describing the Dual Network Simplex method, focusing on the selection rule for the entering arc. The notes explain that the entering arc must connect two subtrees created by removing the leaving arc, and it must do so in the opposite direction of the leaving arc. Additionally, it must have the smallest dual slack among all such arcs. The underlined phrases highlight key aspects of the rule.

Slide 36

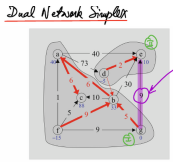
Content

Dual Network Simplex

Description

Slide 36 of 45 in a Linear Programming course illustrates an iteration of the Dual Network Simplex method applied to a network flow problem. The figure shows a network flow graph partitioned into two subtrees after removing a leaving arc (not explicitly shown but implied by the two subtrees). The algorithm identifies the entering arc, which must reconnect the two subtrees and point in the opposite direction of the leaving arc. The entering arc is selected based on the smallest dual value (Z value) among all such arcs, as indicated by the handwritten note. The graph visually represents the state of the algorithm before the pivot operation, showing the current flow values, dual values, and node potentials. The red arrows represent the current flow directions. The purple arc (e-g) represents the entering arc selected based on the minimum dual value. The context of the course (Linear Programming) is crucial for understanding the network simplex algorithm and its application to solving optimization problems. The figure visually represents an intermediate solution obtained during the algorithm's execution, highlighting the current flow values and dual variables. The division into two subtrees is a key characteristic of the dual network simplex method.

Figures



(a) A network flow graph illustrating the Dual Network Simplex method. The graph contains nodes a, b, c, d, e, f, and g, connected by arcs. Each arc has a flow value (black) and a dual value (blue). Node potentials are shown in blue. Red arrows indicate the direction of flow. Nodes a, c, and f form one subtree, while nodes b, d, e, and g form another subtree. A purple arc connects nodes e and g, highlighting the entering arc. The arc values are: a-c (6,6), a-d (6,73), c-b (10,10), b-g (5,5), f-a (1,1), f-c (5,5), d-b (2,-5), e-g (9,9), e-d (2,2), a-e (2,-2). Node values are: a(40), b(33), c(88), d(-5), e(10), f(-15), g(0). A handwritten note indicates "Smallest Z value enters".

Slide 37

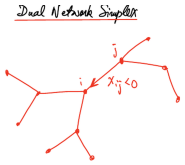
Content

Dual Network Simplex

Description

Slide 37 of 45 in a Linear Programming course illustrates a step in the Dual Network Simplex method. The figure is a hand-drawn network graph representing a spanning tree. The key element is a labeled edge connecting nodes i and j, with the label $x_{ij} < 0$. This indicates that the flow along this arc is negative, violating the non-negativity constraint of the network flow problem. This arc is identified as the leaving arc in the Dual Network Simplex algorithm. The algorithm proceeds by finding an entering arc to replace the leaving arc, maintaining the tree structure and restoring feasibility. The context of the course (Linear Programming) is crucial for understanding the network simplex algorithm and its application to solving optimization problems. The figure visually represents an intermediate solution obtained during the algorithm's execution, highlighting a negative flow value that needs to be resolved. The tree structure is a key characteristic of the network simplex method.

Figures



(a) A hand-drawn network flow graph showing a tree structure with nodes and edges. The nodes are represented by small circles. The edges are represented by lines connecting the nodes. One edge is labeled with an arrow pointing from node i to node j, and the label $x_{ij} < 0$ is written next to it. Nodes i and j are also labeled. The overall structure resembles a tree with several branches. The graph is drawn in red.

Slide 38

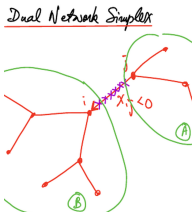
Content

Dual Network Simplex

Description

Slide 38 of 45 in a Linear Programming course illustrates a key step in the Dual Network Simplex method. The figure depicts a network graph partitioned into two disjoint subtrees, A and B, after the removal of a leaving arc (not explicitly shown). A crucial aspect is the presence of an arc with a negative flow value, $x_{ij} < 0$, connecting the two subtrees. This negative flow indicates infeasibility. The purple arc represents the arc with a negative flow value ($x_{ij} < 0$), which is identified as the leaving arc. The algorithm will now select an entering arc to replace the leaving arc, maintaining the tree structure and restoring feasibility. The two subtrees are enclosed in green ovals for clarity. The context of the course (Linear Programming) is crucial for understanding the network simplex algorithm and its application to solving optimization problems. The figure visually represents an intermediate solution obtained during the algorithm's execution, highlighting the negative flow value that needs to be resolved. The tree structure and the identification of the leaving arc are key characteristics of the dual network simplex method.

Figures



(a) A hand-drawn network flow graph showing two disjoint subtrees, labeled A and B, connected by a purple arc. The arc is labeled $x_{ij} < 0$. Nodes in each subtree are connected by red lines. The subtrees are enclosed in green ovals. The labels A and B are written inside the green ovals.

Slide 39

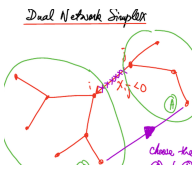
Content

Dual Network Simplex

Description

Slide 39 of 45 in a Linear Programming course illustrates a step in the Dual Network Simplex method for network flow problems. The figure shows a network graph partitioned into two disjoint subtrees, A and B, after the removal of a leaving arc (not explicitly shown). A key feature is the arc with a negative flow value, $x_{ij} < 0$, connecting the two subtrees. This negative flow indicates infeasibility. The purple arc represents the leaving arc. The algorithm proceeds by selecting an entering arc from subtree B to subtree A to replace the leaving arc. The entering arc is chosen based on the smallest slack (not explicitly defined in the figure but implied by the context of the Dual Network Simplex method). The handwritten instruction "Choose the arc from B to A with smallest slack" clarifies the selection criterion for the entering arc. The context of the course (Linear Programming) is crucial for understanding the network simplex algorithm and its application to solving optimization problems. The figure visually represents an intermediate solution obtained during the algorithm's execution, highlighting the negative flow value and the selection process for the entering arc. The tree structure and the identification of the leaving and entering arcs are key characteristics of the dual network simplex method.

Figures



(a) A hand-drawn network flow graph showing two disjoint subtrees, labeled A and B, enclosed in green ovals. The subtrees are connected by a purple arc labeled $x_{ij} < 0$. Nodes within each subtree are connected by red lines. A purple arrow points from subtree B to subtree A, with the handwritten text "Choose the arc from B to A with smallest slack".

Slide 40

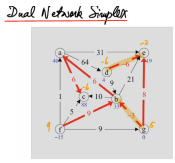
Content

Dual Network Simplex

Description

Slide 40 of 45 in a Linear Programming course shows an intermediate solution of the Dual Network Simplex method applied to a network flow problem. The figure is a network graph representing a spanning tree with nodes labeled a through g, each with a supply/demand value. Edges are labeled with flow values; some are positive, and some are negative. The negative flow values (−6 on edges cd and de, and −6 on edge ce) indicate infeasibility. The red highlighted edges represent the current basic feasible solution (after the first pivot). The algorithm aims to resolve the infeasibility by finding an entering arc to replace a leaving arc (the arc with a negative flow). The context of the course (Linear Programming) is crucial for understanding the network simplex algorithm and its application to solving optimization problems. The figure visually represents an intermediate solution obtained during the algorithm's execution, highlighting negative flow values that need to be resolved. The tree structure is a key characteristic of the network simplex method. The caption "FIGURE 14.15. The tree solution after the first pivot" indicates that this is the state of the network after the first iteration of the Dual Network Simplex algorithm.

Figures



(a) A network flow graph showing nodes a, b, c, d, e, f, g connected by edges. Node a has value 40, b has value 33, c has value 88, d has value 4, e has value 19, f has value -15, g has value 0. Edges have values: af=5, ab=6, ac=1, ad=64, ae=31, bc=10, bd=9, be=21, cd=-6, ce=-6, de=-6, df=9, eg=8, fg=9. Edges af, ab, bc, be, eg are highlighted in red, indicating flow. Edge cd has a negative value, indicating infeasibility. The figure is labeled "FIGURE 14.15. The tree solution after the first pivot."

Slide 41

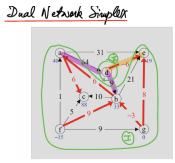
Content

Dual Network Simplex

Description

Slide 41 of 45 in a Linear Programming course depicts an intermediate solution within the Dual Network Simplex method applied to a network flow problem. The figure, labeled "FIGURE 14.15. The tree solution after the first pivot," shows a network graph with nodes a through g, each possessing a supply/demand value. Edges are annotated with flow values; some are positive, others negative. The negative flow values (-6 on edges cd, ce, and de) signify infeasibility in the current solution. Red highlighted edges (af, ab, bc, be, eg) represent the current basic feasible solution (after the first pivot). The algorithm aims to resolve the infeasibility by identifying an entering arc to replace a leaving arc (an arc with negative flow). The context of the course (Linear Programming) is crucial for understanding the network simplex algorithm and its application to optimization problems. The figure visually represents an intermediate solution, highlighting the negative flow values requiring resolution. The tree structure is a key characteristic of the network simplex method. The caption indicates this is the network's state after the first iteration of the Dual Network Simplex algorithm.

Figures



(a) A network flow graph showing nodes a, b, c, d, e, f, g connected by edges. Node a has value 40, b has value 33, c has value 88, d has value 4, e has value 19, f has value -15, g has value 0. Edges have values: af=5, ab=6, ac=1, ad=64, ae=31, bc=10, bd=9, be=21, cd=-6, ce=-6, de=-6, df=9, eg=8, fg=9. Edges af, ab, bc, be, eg are highlighted in red, indicating flow. Edge cd has a negative value, indicating infeasibility. The figure is labeled "FIGURE 14.15. The tree solution after the first pivot."

Slide 42

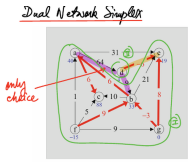
Content

Dual Network Simplex

Description

Slide 42 of 45 in a Linear Programming course illustrates an intermediate step in the Dual Network Simplex method. The figure, labeled "FIGURE 14.15. The tree solution after the first pivot," shows a network flow graph with nodes a through g, each having a supply/demand value. Edges are labeled with flow values; some are positive, and some are negative. The negative flow values on edges cd, ce, and de indicate infeasibility in the current solution. Red edges (af, ab, bc, be, eg) represent the current basic feasible solution after the first pivot. The algorithm aims to resolve the infeasibility by selecting an entering arc to replace a leaving arc (an arc with negative flow). A handwritten red arrow points to the graph with the text "only choice," suggesting that there is only one possible choice for the entering arc in this specific iteration of the algorithm. The context of the course (Linear Programming) is crucial for understanding the network simplex algorithm and its application to optimization problems. The figure visually represents an intermediate solution, highlighting the negative flow values and the unique choice for the entering arc. The tree structure is a key characteristic of the network simplex method. The caption indicates this is the network's state after the first iteration of the Dual Network Simplex algorithm.

Figures



(a) A network flow graph showing nodes a, b, c, d, e, f, g connected by edges. Node a has value 40, b has value 33, c has value 88, d has value 4, e has value 19, f has value -15, g has value 0. Edges have values: af=5, ab=6, ac=1, ad=64, ae=31, bc=10, bd=9, be=21, cd=-6, ce=-6, de=-6, df=9, eg=8, fg=9. Edges af, ab, bc, be, eg are highlighted in red, indicating flow. Edge cd has a negative value, indicating infeasibility. A red arrow points to the graph from the left with the handwritten text "only choice". The figure is labeled "FIGURE 14.15. The tree solution after the first pivot."

Slide 43

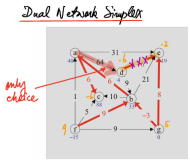
Content

Dual Network Simplex

Description

Slide 43 of 45 in a Linear Programming course shows a network flow graph representing an intermediate solution in the Dual Network Simplex method. The graph depicts nodes (a-g) with supply/demand values and edges with flow values. Negative flows (-6 on edges cd, ce, de) indicate infeasibility. Red edges (af, ab, bc, be, eg) represent the current basic feasible solution after a pivot. A handwritten red arrow points to the graph with the text "only choice," indicating a unique choice for the entering arc in this iteration to resolve the infeasibility. The figure is labeled "FIGURE 14.15. The tree solution after the first pivot," showing the state after the first iteration of the Dual Network Simplex algorithm. The context of Linear Programming is crucial for understanding the algorithm's application to optimization problems. The tree structure is a key feature of the network simplex method. The slide visually represents an intermediate solution, highlighting the infeasibility and the unique choice for the entering arc in the next iteration.

Figures



(a) A network flow graph showing nodes a, b, c, d, e, f, g connected by edges. Node a has value 40, b has value 33, c has value 88, d has value 4, e has value 19, f has value -15, g has value 0. Edges have values: af=5, ab=6, ac=1, ad=64, ae=31, bc=10, bd=9, be=21, cd=-6, ce=-6, de=-6, df=9, eg=8, fg=9. Edges af, ab, bc, be, eg are highlighted in red, indicating flow. Edge cd has a negative value, indicating infeasibility. A red arrow points to the graph from the left with the handwritten text "only choice". The figure is labeled "FIGURE 14.15. The tree solution after the first pivot."

Slide 44

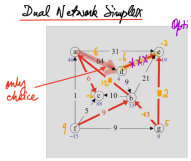
Content

Dual Network Simplex

Description

Slide 44 of 45 in a Linear Programming course displays the final iteration of the Dual Network Simplex method applied to a network flow problem. The figure, labeled "FIGURE 14.15. The tree solution after the first pivot," shows a network graph with nodes a through g, each with a supply/demand value (shown in blue). Edges are annotated with flow values; some are positive, others negative (shown in purple). The negative flow values indicate infeasibility in previous iterations. Red highlighted edges represent the current basic feasible solution. A handwritten red arrow points to the graph with the text "only choice," indicating a unique choice for the entering arc in a previous iteration. The handwritten "Optimal" in purple signifies that the algorithm has converged to an optimal solution. The values on the edges are shown in different colors to highlight the changes during the iterations. The context of the course (Linear Programming) is crucial for understanding the network simplex algorithm and its application to optimization problems. The figure visually represents the final optimal solution, highlighting the flow values on each edge. The tree structure is a key characteristic of the network simplex method. The caption indicates this is the network's state after the final iteration of the Dual Network Simplex algorithm.

Figures



(a) A network flow graph showing nodes a, b, c, d, e, f, g connected by edges. Node a has value 40, b has value 33, c has value 88, d has value 4, e has value 19, f has value -15, g has value 0. Edges have values: af=5, ab=6, ac=1, ad=64, ae=31, bc=10, bd=9, be=21, cd=-6, ce=-6, de=-6, df=9, eg=8, fg=9. Edges af, ab, bc, be, eg are highlighted in red, indicating flow. Edge cd has a negative value, indicating infeasibility. A red arrow points to the graph from the left with the handwritten text "only choice". The edges de, cd, ce have negative values (-6) written in purple. The edges ad, ae, be have values (64, 31, 21) written in red. The edges ab, bc, eg have values (6, 10, 8) written in red. The edge fg has value 9 written in black. The edge af has value 5 written in black. The edge bd has value 9 written in black. The edge ac has value 1 written in black. The nodes have values (40, 33, 88, 4, 19, -15, 0) written in blue. There is a handwritten "Optimal" in purple in the upper right corner. The figure is labeled "FIGURE 14.15. The tree solution after the first pivot."

Slide 45

Content

5. Putting It All Together

As we saw in Chapter 5, for linear programming the primal and the dual simplex methods form the foundation on which one can build a few different variants of the simplex method. The same is true here in the context of network flows.

For example, one can build a two-phased procedure in which one first uses the dual network simplex method (with costs artificially and temporarily altered to ensure dual feasibility of an initial tree solution) to find a primal feasible solution and then uses the primal network simplex method to move from the feasible solution to an optimal one.

Alternatively, one can use the primal network simplex method (with supplies temporarily altered to ensure primal feasibility of an initial tree solution) to find a dual feasible solution and then use the dual network simplex method (with the original supplies) to move from the dual feasible solution to an optimal one.

Finally, as described for linear programming in Chapter 7, one can define a parametric self-dual method in which primal pivots and dual pivots are intermingled as needed so as to reduce a perturbation parameter μ from ∞ to zero.

Description

Slide 45 of 45 summarizes different approaches to solving network flow problems using primal and dual simplex methods within the context of a Linear Programming course. The slide outlines three main strategies: a two-phased approach (using dual then primal simplex), an alternative approach (using primal then dual simplex), and a parametric self-dual method. Each strategy involves modifying the problem's parameters (costs or supplies) to ensure feasibility before applying the appropriate simplex method. The mention of Chapter 5 and Chapter 7 indicates that these chapters covered the foundational simplex methods and the parametric self-dual method, respectively. The use of μ (mu) represents a perturbation parameter, a common technique in optimization to handle degeneracy or infeasibility. The overall message is that the primal and dual simplex methods are versatile building blocks for creating more sophisticated algorithms to solve network flow problems.

Figures


5. Putting It All Together

As we saw in Chapter 5, for linear programming the primal and the dual simplex methods form the foundation on which one can build a few different variants of the simplex method. The same is true here in the context of network flows.

For example, one can build a two-phased procedure in which one first uses the dual network simplex method (with costs artificially and temporarily altered to ensure dual feasibility of an initial tree solution) to find a primal feasible solution and then uses the primal network simplex method to move from the feasible solution to an optimal one.

Alternatively, one can use the primal network simplex method (with supplies temporarily altered to ensure primal feasibility of an initial tree solution) to find a dual feasible solution and then use the dual network simplex method (with the original supplies) to move from the dual feasible solution to an optimal one.

Finally, as described for linear programming in Chapter 7, one can define a parametric self-dual method in which primal pivots and dual pivots are intermingled as needed so as to reduce a perturbation parameter μ from ∞ to zero.

(a) Text describing different variants of the simplex method for linear programming and network flows. The text is organized into paragraphs, with key phrases highlighted implicitly by their placement and formatting.